Resolve Once, Route Always: Content-Addressed Script Environments versus Per-Job Dependency Resolution

Resolve Once, Route Always: Content-Addressed Script Environments versus Per-Job Dependency Resolution

*Research companion to the Facetwork thesis (`thesis.md`). Empirical basis: a measured comparison against Ray's `runtime_env` on a single host, plus the environment machinery's own bake/lazy paths; all code, manifests and timings are reproducible from the Facetwork repository. Spark and Dask are discussed as related designs but were **not** measured — see §6 and §7.*

Abstract

A distributed task runtime that executes user code must answer where that code's dependencies come from. The common answer is **per-job resolution**: the job carries a dependency specification, and the system resolves and materializes it when the job runs — Ray's `runtime_env`, Spark's archive shipping, Dask's worker plugins. We describe an alternative built into a leaderless workflow runtime: dependencies are declared in the workflow language, **resolved once at publish time** into a pinned manifest, **content-addressed** by a hash of that manifest, and the hash becomes a *routing key* — a task is claimed only by a runner that already advertises the environment, rather than materialized on whichever node happens to pick it up. Environments are provisioned in three tiers: baked into the image, materialized lazily on demand, or provided out-of-band by a foreign runtime. We report a measured comparison with Ray on one host. The results are mixed and we report them as such: moving provisioning to build time makes first-execution latency ~6x lower than Ray's cold path (2.6s vs 15.2s), Ray's warm path is ~4x faster than ours (0.63s vs 2.6s) for reasons that have nothing to do with dependency management, and our *lazy* tier is the weakest path measured — slower than Ray's cold install (34s vs 15.2s) because it pays a demand-scan cadence and a re-claim that Ray's in-process install does not. The durable difference is not latency but **when resolution happens and what is recorded**: an unpinned `humanize` freezes to `humanize==4.16.0` under a stable hash at publish, so a re-run a year later installs what it was published with, while a per-job spec re-resolves to whatever the index serves that day.

1. Introduction

Facetwork workflows are written in a DSL (FFL) and executed by a leaderless fleet of interchangeable runners. A workflow step may be a `script` block — user code that runs in-process on whichever runner claims it. That immediately raises the dependency question: the script needs libraries the runner may not have.

The mechanisms in wide use resolve dependencies **per job**. Ray's `runtime_env={"pip": [...]}` builds a virtualenv on the node when the job first needs it. Spark ships an archive or conda pack with the application. Dask installs via worker plugins or expects a pre-built worker image. All are reasonable, and all share two properties: the specification is resolved at *execution* time, and it is materialized *wherever the work landed*.

We took a different position, and this paper reports what it bought and what it cost:

1. **Declared in the language.** An `environment` is an FFL declaration (language + `requires`), and a script-bearing facet names it via `in environment`.
2. **Resolved once, at publish.** Publication resolves `requires` into exact pins and content-addresses the result: `manifest_hash = sha256(language, sorted pins)[:16]`.
3. **The hash is a routing key.** Script tasks carry it; runners advertise the hashes they provide; the claim query filters on it. Work goes to an environment rather than an environment being built for the work.
4. **Three provisioning tiers.** *Baked* (built into the runner image), *lazy* (materialized on demand, then re-advertised), *provided* (a foreign runtime supplies it, used for the JVM agents).

Contributions: the design (§2), a measured comparison against the closest prior art (§4), an honest account of where it loses (§5), and the reproducibility property that motivates the whole thing (§3).

2. Design

Freeze. At publish, each `environment` declaration is resolved. `pip install --dry-run --report` performs full resolution without installing; already-exact specs pass through without touching the network. The result is a manifest — `{language, requires, pins, resolved}` — and its identity is the hash of `{language, sorted(pins)}`. The loose `requires` are provenance; only what actually runs participates in identity.

Route. A script task carries `environment_hash`. `claim_task` filters on the claiming runner's advertised set, so a runner without the environment never takes the task — the same both-sides-of-the-queue discipline the runtime uses for handler names and AST features. An untagged task stays claimable by everyone.

Provision. *Baked*: an image-build step sweeps the FFL sources, freezes every declaration and materializes the python ones into `FW_ENV_ROOT`; a materialization failure fails the build, because a half-provided image would advertise nothing and strand its tasks silently. *Lazy*: a runner periodically scans for demanded-but-unprovided hashes, builds the venv, re-registers, and becomes eligible. *Provided*: foreign-language environments are advertised by out-of-band agents and never built by us.

3. The property this is for: resolve once

The reproducibility claim is not a benchmark result, so we state it as a demonstration. Freezing the deliberately-unpinned declaration `requires = ["humanize"]`:

```
run 1 pins: ['humanize==4.16.0']   hash: 26d5d2c949711b88
run 2 pins: ['humanize==4.16.0']   hash: 26d5d2c949711b88
```

The workflow records the hash. Re-running it in a year installs `4.16.0`, because that is what the manifest says — not whatever `humanize` resolves to then. The equivalent Ray job, `runtime_env={"pip": ["humanize"]}`, re-resolves at every submission; pinning is available and encouraged, but it is the *user's* discipline rather than a property of the system, and an unpinned spec is accepted silently. The difference is where the burden of reproducibility sits.

This is the same argument the catalog makes for `use` resolution, applied to dependencies: a published artifact should record what it ran with.

4. Measurement

Setup. One host (Apple Silicon, 14 GB VM for containers), Ray 2.57.0 in an isolated virtualenv, Facetwork at the deployed build. The workload is the minimum that isolates provisioning: a task that imports a third-party package (`humanize`) and formats a number. We measure **time from submission to first result** — the quantity a user waits for.

Conditions. *Ray cold*: a freshly started cluster and no cached runtime resources for the spec. *Ray warm*: the same long-lived cluster, same spec, subsequent submissions — the fair analogue of Facetwork's long-lived runners. *Facetwork baked*: the environment is in the runner image (the fleet's 15 runners advertise its hash). *Facetwork lazy*: a pin no runner provides (`humanize==4.11.0`, `4.10.0`, `4.9.0`), forcing on-demand materialization.

Condition	Samples (s)	Median
Ray `runtime_env`, cold	11.9, 16.5, 17.3	**16.5**
Ray `runtime_env`, warm	0.63, 0.63	**0.63**
Facetwork, baked	11.0 (first), 2.5, 2.7	**2.6** (steady)
Facetwork, lazy	25.3, 21.8, 55.7	**25.3**

Three readings, in decreasing order of how flattering they are:

Baking beats cold resolution (2.6s vs 16.5s). This is the intended result and the only one that is really about dependency *management*: the ~15s Ray spends installing on first use is work Facetwork did at image build, so it is not on the user's clock at all.

Ray warm beats baked (0.63s vs 2.6s), and this is not a dependency result. Once Ray has the env cached, dispatch is an in-process remote call. Facetwork's 2.6s is workflow-engine overhead — bootstrap task, persisted step rows, task creation, a 1s claim poll — paid by every step regardless of environments. A reader should take from this that our *engine* is heavier, not that our environment model is slower. Comparing a workflow engine's end-to-end step latency with a task API's is apples to oranges; we report it because omitting it would be the more misleading choice.

Lazy is our weakest path (25–56s vs Ray's 16.5s cold). Ray's on-demand install is simply better engineered than ours for the one-off case. Our lazy tier pays a demand-scan cadence before it notices the need, then materializes, re-registers, and waits to re-claim; the 55.7s outlier is a run that missed a scan window. Ray installs inline on the node that already has the task. If the lazy tier were the main path, this comparison would be a straightforward loss.

5. What the measurements do and do not support

They support a narrow claim: **moving resolution to publish and provisioning to build time removes it from execution latency**, and content-addressing makes it safe to do so, because the thing built at bake time is provably the thing the workflow asked for. They also support a criticism of our own lazy tier.

They do not support a general "faster than Ray" claim, and we make none. On the steady-state path Ray is several times quicker for reasons unrelated to this paper's subject. Nor do they say anything about throughput, scheduling, or scale — this is a latency-of-first-execution measurement on one host with one tiny dependency. A large dependency set (`torch`, `rasterio`) would widen every provisioning number and probably widen the baked-vs-cold gap, but we did not measure it and do not claim it.

6. Related designs

Ray `runtime_env` is the closest prior art and the only system we measured. It supports pinned specs, per-actor and per-task envs, and caches materialized envs per node — the caching is why the warm number is what it is. The design difference is not caching but *identity*: Ray's cache key is derived from the spec as written, whereas ours is derived from the resolved answer, which is also what routes the work.

Spark ships dependencies with the application (`--archives`, conda-pack). That is closer to our *baked* tier in spirit — provisioning precedes execution — but the unit is the application submission rather than a content-addressed artifact reusable across submissions.

Dask typically expects worker images to carry dependencies, with `upload_file`/plugins for smaller cases; the "make the worker right beforehand" posture again resembles baking, without a manifest identity.

Nix and content-addressed builds are where the identity idea comes from, and the honest comparison is unflattering in one respect: Nix hashes the full build closure, while our hash covers `{language, pins}` and trusts the interpreter and wheels beneath it. Two runners with the same hash and different Python patch versions are, to us, the same environment. That is a deliberate scope limit (§7), not an oversight.

Neither Spark nor Dask was benchmarked here. Their provisioning models differ enough that a fair comparison needs care we did not take, and reporting numbers we did not measure would be worse than reporting none.

7. Threats to validity and scope

One host, one dependency, n=3 per condition, medians reported rather than distributions. The comparison is between a *workflow engine* and a *task API*, so the absolute latencies are not like-for-like; only the within-system comparisons (baked vs lazy, cold vs warm) are clean.

The hash covers `{language, pins}` only. It does not capture the interpreter version, the platform, or the wheels' own transitive native libraries, so two hosts can advertise the same environment and differ underneath. `python = "3.12"` as a manifest field was considered and deferred; until it exists, the identity is weaker than the content-addressing language suggests.

Lazy materialization's cadence is a knob we have not tuned, and the 55.7s outlier suggests the tuning matters more than we assumed. We have not measured the *provided* (foreign-language) tier at all — it is exercised by the JVM agents in production but has no benchmark here.

Finally, this is a report on a design we built and like. The most valuable external work would be a fair Spark/Dask/Ray comparison run by someone with no stake in the outcome, on a dependency set large enough for provisioning to dominate.

8. Conclusion

Per-job dependency resolution asks the runtime to answer "what does this code need?" at the moment it is least convenient — when the work is already scheduled. Declaring environments in the workflow language, resolving them once at publish, and content-addressing the answer moves that question to a point where it can be answered slowly and recorded permanently; making the resulting hash a routing key means work is sent to an environment rather than an environment being built for the work.

The measured payoff is exactly one thing: provisioning disappears from execution latency when the environment is baked (2.6s vs 16.5s). The measured cost is also one thing: our lazy tier, the fallback for anything not baked, is slower than the per-job install it was meant to improve on. The property we actually built it for is not in the timing table at all — an unpinned dependency freezes to a pin, under a hash the workflow carries, so that running it again means running the same thing again.